Introduction to Artificial Intelligence - Final Project

David Avni

February 2026

Project Links:

GitHub Repository (Source Code) — Google Colab (Neural Network)

1 Part I: Regression Task - Fuel Efficiency Prediction

1.1 Exploratory Data Analysis (EDA)

The Auto MPG dataset was analyzed to understand the relationships between vehicle characteristics and fuel efficiency. During the initial analysis, 6 missing values were identified in the 'horsepower' feature. These were handled using median imputation to maintain the dataset's integrity without introducing significant bias. A strong negative correlation was observed between vehicle weight and MPG, suggesting a linear or near-linear relationship.

1.2 Preprocessing

Before modeling, all features were standardized using `StandardScaler`. This step is critical because the features have different scales (e.g., weight in the thousands, acceleration in the tens). Standardization ensures that all features contribute equally to the distance metrics used in algorithms like KNN and helps gradient-based optimizers converge faster.

1.3 Linear Regression Baseline

A simple linear regression model was trained as a baseline. The performance metrics on the test set were:

- **MSE:** 8.20
- **RMSE:** 2.86
- **MAE:** 2.26
- R^2 **Score:** 0.85

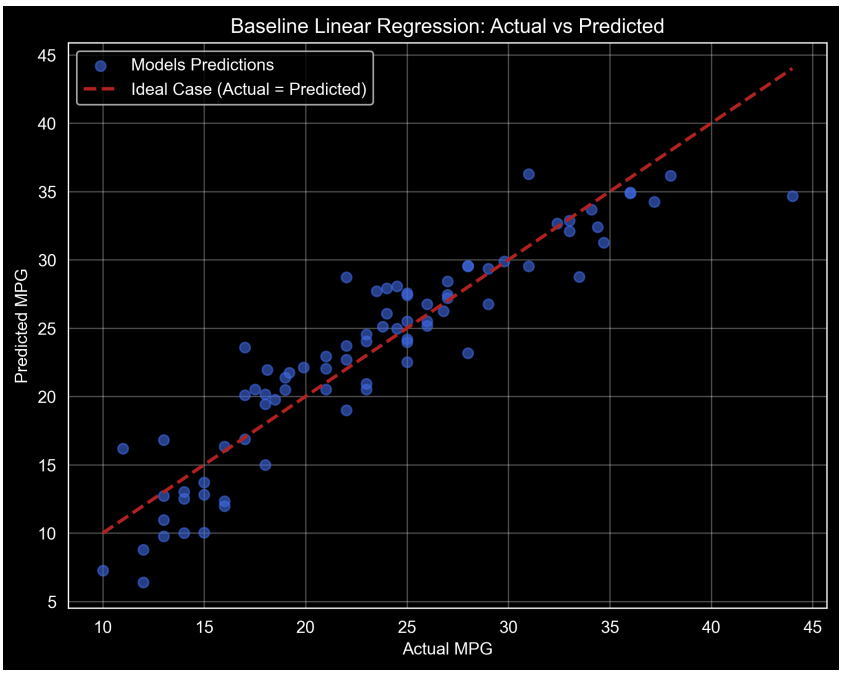

Figure 1: Actual vs. Predicted MPG (Linear Regression Baseline)

The "Actual vs. Predicted" plot shows that while the model captures the general trend, residuals tend to increase at the extreme ends of the MPG range, suggesting non-linear patterns.

1.4 Polynomial Regression and Model Complexity

To capture non-linearities, polynomial features were introduced. We evaluated degrees 1 through 5 to analyze the bias-variance tradeoff.

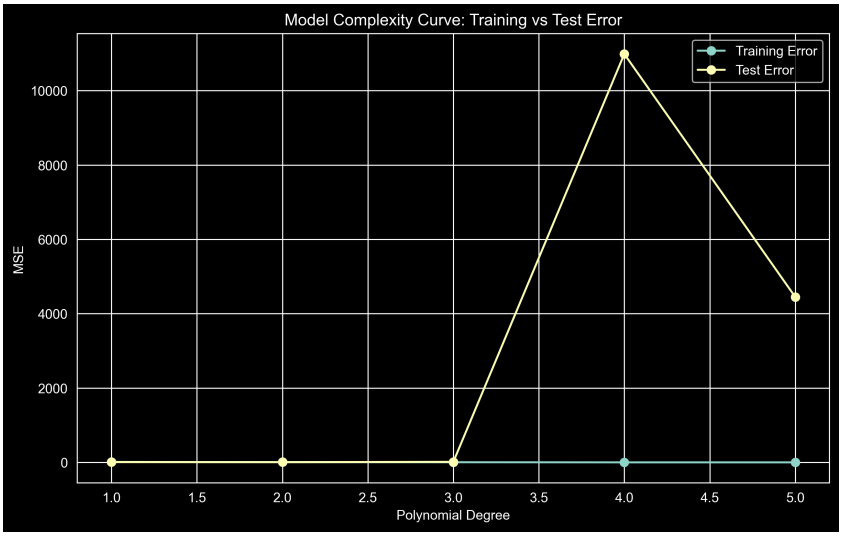

Figure 2: Model Complexity Curve: Training vs. Test MSE

The results showed that **Degree 2** provided the optimal balance, reducing the Test MSE to 5.97. Degrees 3 and above exhibited severe overfitting, where the training error reached near-zero while the test error increased exponentially.

1.5 K-Nearest Neighbors (KNN) Regression

A non-parametric approach was tested using KNN. We conducted a grid search for the optimal k.

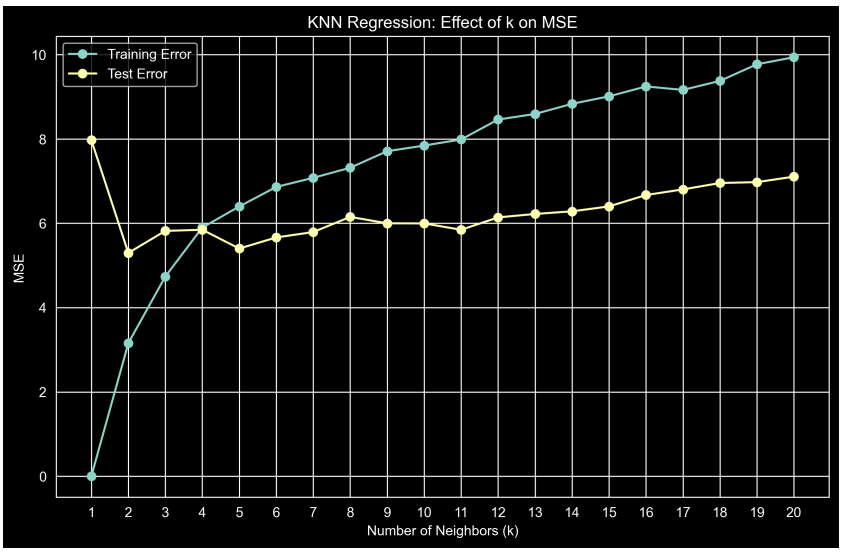

Figure 3: Effect of K-Neighbors on Model Performance (MSE)

The optimal number of neighbors was found to be $k = 2$, achieving an MSE of **5.30**. This model provided the best performance among all tested methods, successfully capturing local non-linearities in the data.

1.6 Optimization Behavior

The behavior of Stochastic Gradient Descent (SGD) was analyzed using two different learning rates (η).

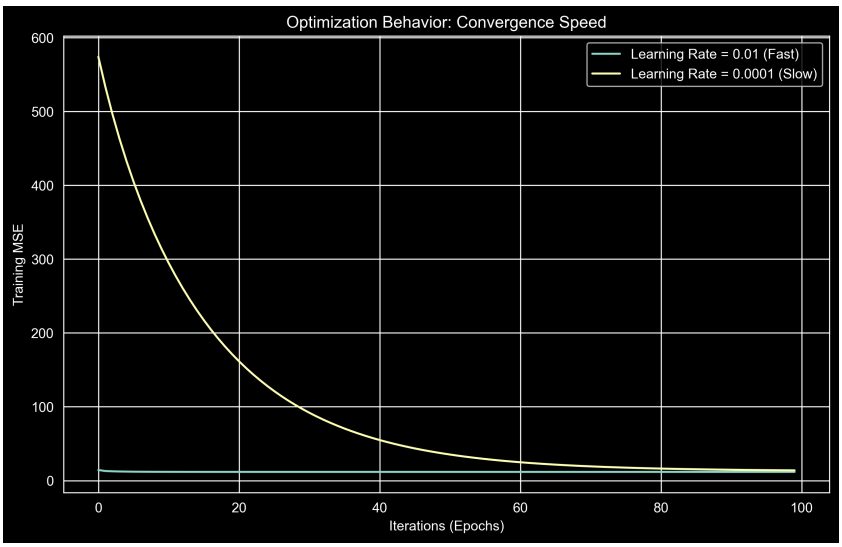

Figure 4: Optimization Convergence: Training MSE vs. Iterations

As illustrated, a higher learning rate ($\eta = 0.01$) led to rapid convergence but with visible oscillations. A lower rate ($\eta = 0.0001$) was stable but required significantly more iterations to converge.

1.7 Conclusion

The **KNN (k=2)** model was selected as the final model for this task, as it demonstrated the lowest generalization error on the held-out test set.

2 Part II: Classification Task - CIFAR-10

2.1 Experimental Results

I compared three classical models for this multi-class classification task on the CIFAR-10 dataset:

- **KNN (k=5):** Achieved an accuracy of **27.32%**. This was the lowest performing model, as pixel-wise distance metrics are often insufficient for high-dimensional image data.
- **Logistic Regression:** Achieved an accuracy of **31.06%**.
- **SVM (RBF Kernel):** Achieved the highest performance with **44.35%** accuracy.

2.2 Error Analysis (Confusion Matrix)

The Confusion Matrix (Figure 5) provides deeper insight into the model's behavior. We can observe high values along the diagonal, representing correct predictions. However, significant confusion exists between visually similar classes. For example, 'Cats' are frequently misclassified as 'Dogs', and 'Automobiles' are confused with 'Trucks'.

Figure 5: Confusion Matrix for the SVM Model on CIFAR-10

3 Part III: Neural Network Classification (PyTorch)

3.1 Architectural Design

For the final task, I designed a Convolutional Neural Network (CNN) using the PyTorch Sequential API. The architecture consists of:

- **Three Convolutional Layers:** 32, 64, and 128 filters respectively, each followed by Batch Normalization and ReLU activation.
- **Max-Pooling Layers:** Used after each convolution to focus on high-level features.
- **Fully Connected Layers:** A 512-unit hidden layer with **Dropout (0.3)** to prevent overfitting, followed by the final 10-class output layer.

3.2 Hyperparameter Selection

To select the optimal hyperparameters as required by the assignment guidelines, I performed a systematic comparison between different learning rates (10^{-3} and 10^{-4}). While both showed a downward trend in loss, the learning rate of 10^{-3} demonstrated significantly faster convergence in the initial epochs and was thus selected for the final 10-epoch training session.

3.3 Performance and Comparison

The CNN achieved a peak validation accuracy of **80.20%**. This represents a massive improvement over classical methods:

Model	**Final Accuracy**
KNN (Part II)	27.32%
Logistic Regression (Part II)	31.06%
SVM (RBF Kernel) (Part II)	44.35%
CNN (PyTorch)	**80.20%**

Table 1: Comparative performance of all models on CIFAR-10

3.4 Conclusion

The significant leap from 44.35% (SVM) to 80.20% (CNN) demonstrates the power of Deep Learning in computer vision. While classical models struggle with high-dimensional pixel data, the CNN effectively learns hierarchical feature representations (edges, shapes, and complex patterns).